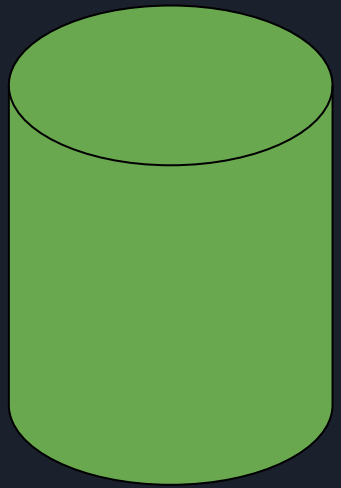
GESTIÓN DE DATOS

PL/SQL - MANEJO DE EXCEPCIONES

RECORDERIS...



¿QUÉ APRENDEREMOS HOY?



- Entendimiento de los handlers (continue y exit)
- Códigos de error VS sqlstate
- Diferenciar entre excepciones cliente VS excepciones servidor

Pregunta: ¿Qué es una excepción?



Y entonces...¿Qué es un error?



Referencia de excepciones

MySQL:
https://dev.mysql.com/doc/refman/8.0/en/error-message-elements.html

MariaDB:
https://mariadb.com/kb/en/mariadb-error-code-reference/

Referencia de excepciones

SQLSTATE:

- Códigos estándar entre varios DBMS (léase ANSI SQL y ODBC)
- Alfanumérico (XXYYY)

ERROR CODE:

- Códigos específicos para MySQL (en ciertos casos MariaDB)
- Numérico

Referencia de excepciones

SQLSTATE (XXYYY):

- XX
  - 00: Exitoso
  - 01: Advertencia
  - 02: Datos no encontrados
  - > 03: Excepción
- HY000: Errores de cliente

ERROR CODE:

- 1~999: Globales (tanto de cliente como de servidor)
- 1000~1999: Mensajes desde el servidor hacia los clientes
- 2000~2999: Errores cliente
- 3000~4999: Errores servidor
- 5000~5999: Extensiones
- 10000~49999: Sólo servidor
- 50000~51999: Terceros

Excepción VS Error

Excepción:

- **Si el usuario puede provocarlo**
- **Si el desarrollador omitió validaciones**
- **Se maneja, no se arregla**

Error:

- **No se puede controlar (o es muy complejo hacerlo)**

DEFINICIÓN: HANDLER

👉 Estructura que permite realizar manejo de excepciones en pl/sql.

👉 Regla de reacción ante un evento. Permite decidir si continuar o parar.

👉 Es el equivalente al "try-catch-finally" o "try-catch-except" usado en algunos lenguajes de programación

DEFINICIÓN: HANDLER

✅ "continue handler":

Maneja la excepción y luego la ejecución continúa
Ejemplo: Si no hay datos, retornar N/A

❌ "exit handler":

Maneja la excepción y la ejecución se detiene
Ejemplo: Si no hay datos, no tiene sentido continuar

DEFINICIÓN: SIGNAL

👉 Equivalente al "throw" en Java, o al "raise" en Python

👉 Permite retornar un mensaje personalizado al backend.

👉 Ejemplo: En vez de "no data found", se retorna "Error 50001: Cliente no existe en el sistema"

Ejemplo: excepciones personalizadas

"signal" es el equivalente al "throws" o "raise" en otros lenguajes.

Se debe especificar "sqlstate", "***mysql_errno***" y un mensaje personalizado en "***message_text***", ***obligatoriamente en variables con estos nombres***

```
create function fn_division_01(
   p_number_1 int,
   p_number_2 int
)
returns real
begin

   if p_number_2 = 0 then
      signal sqlstate 'DZ001'
          set mysql_errno  = 50001,
              message_text = 'Ops! División por 0';
   end if;

   return p_number_1 / p_number_2;
end;
```

Ejemplo: handler tipo "continue"

"Continue" significa que ocurre la excepción, se hace el manejo como se especifica en el handler, y la ejecución continúa

Si este código genera excepción, se invoca lo que esté en el handler

Como el handler es tipo "continue", esta línea se ejecuta sin inconvenientes

```
create function fn_get_singer_by_id(
    p_id  type of singers.id
)
returns varchar(100)
begin
    declare v_full_name varchar(100) default '';
    declare c_singers cursor for
        select  concat(first_name, ' ', last_name)
        from    singers
        where   id = p_id;
    declare continue handler for not found
        set v_full_name := 'N/A';

    open   c_singers;
    fetch  c_singers
    into   v_full_name;
    close  c_singers;

    return v_full_name;
end;
```

"not found" es un tipo de manejador para cuando se intenta recorrer una estructura que no tiene datos; por ejemplo, un cursor que retorna 0 registros

Ejemplo: handler tipo "exit"

"Exit" significa que ocurre la excepción, se hace el manejo como se especifica en el handler, y la ejecución se detiene

Si este código genera excepción, se invoca lo que esté en el handler

Como el handler es tipo "exit", esta línea nunca se ejecutará

```
create function fn_get_singer_by_id(
    p_id  type of singers.id
)
returns varchar(100)
begin
    declare v_full_name varchar(100) default '';
    declare c_singers cursor for
        select  concat(first_name, ' ', last_name)
        from    singers
        where   id = p_id;
    declare exit handler for not found
        set v_full_name := 'No existe';

    open   c_singers;
    fetch  c_singers
    into   v_full_name;
    close  c_singers;

    return v_full_name;
end;
```

Ejemplo: handler con sqlstate

```
create function fn_get_singer_by_id(
   p_id  type of singers.id,
   p_pad int
)
returns varchar(100)
begin
   declare v_full_name varchar(100) default '';
   declare c_singers cursor for
       select  lpad(first_name, p_pad, '-')
       from    singers
       where   id = p_id;
   declare continue handler for sqlstate '22001'
      set v_full_name = 'Ops, se desbordó!';

   open   c_singers;
   fetch  c_singers
   into   v_full_name;
   close  c_singers;

   return v_full_name;
end;
```

Si ocurre excepción, la variable tendrá este texto

Siempre, haya excepción o no, se retornará esta variable, puesto que el handler es tipo "continue"

El código 22001 se refiere a datos más largos de lo que una variable puede soportar

Ejemplo: handler con error code

```
create function fn_get_singer_by_id(
   p_id  type of singers.id,
   p_pad int
)
returns varchar(100)
begin
   declare v_full_name varchar(100) default '';
   declare c_singers cursor for
       select  lpad(first_name, p_pad, '-')
       from    singers
       where   id = p_id;
   declare continue handler for 1326
       set v_full_name = 'Cerrando un cursor que no se ha abierto!!';

   open   c_singers;
   fetch  c_singers
   into   v_full_name;
   close  c_singers;
   close  c_singers;
   return v_full_name;
end;
```

Esta línea generará excepción y ejecutará el código del handler

El código 1326 se refiere a cerrar cursores ya cerrados

EJERCICIO (1/4)

Modificar el procedimiento almacenado "pr_add_country" creado en clases anteriores.

Se debe tener un manejo de excepciones para cuando se haga una inserción que esté violando la clave primaria. Es decir, cuando se esté insertando un país que ya existe.

Guía: Investigar el código sqlstate 23000 en la documentación de excepciones

EJERCICIO (2/4)

Modificar el procedimiento almacenado "pr_add_city" creado en clases anteriores.

Se debe tener un manejo de excepciones para cuando se haga una inserción que esté violando la clave foránea del ID del país. Es decir, cuando se inserte una ciudad asociada a un país que no existe.

EJERCICIO (3 / 4)

Escribir un procedimiento con las siguientes características:

- Nombre: pr_add_language
- Parámetros de entrada: id del país (p_country_id), nombre del idioma (p_language), si es oficial (p_official).
- Parámetros de salida: cantidad de registros insertados (p_records)
- Descripción: Insertar una nueva relación país/idioma en la tabla countrylanguage. El valor del porcentaje debe ser 0.
- Se debe hacer un manejo de excepciones para cuando se inserte un nombre de idioma que supere los 30 caracteres, o un valor diferente a "T" ó "F" en la oficialidad del idioma.

EJERCICIO (4/4)

Modificar la función/procedimiento creados en clases anteriores que permite consultar la información de un país enviando como parámetro de entrada el ID.

Si el código utiliza open/fetch/close, se debe realizar un manejo de excepciones para cuando se busque un id de país que no existe

¿PREGUNTAS?

